DRS MC508 – SS 2025
Lab Report Exercises 1–8

Simone B. Feldbacher

February 17, 2026

Abstract

The Course Distributed Real Time Systems focuses on the principles and technologies for building distributed real-time systems. The course covers fundamental topics like worst-case execution time analysis, real-time communication, scheduling algorithms, resources, and comunication. These concepts are applied using QNX 8 on a raspberry Pi 4. This report summarizes the 8 assignments completed during the course, highlighting the key concepts and techniques learned in each chapter.

Contents

1 Assignment 1 — WCET **3**
- 1.1 Assignment . . . 3
- 1.2 Implementation . . . 3
  - 1.2.1 Initialization . . . 4
  - 1.2.2 Data Acquisition . . . 4
  - 1.2.3 Calculation . . . 4
  - 1.2.4 State Transition . . . 5
  - 1.2.5 Challenges . . . 6
- 1.3 Evaluation . . . 6

2 Assignment 2 — Scheduling policies **7**
- 2.1 Assignment . . . 7
- 2.2 Approach . . . 7
- 2.3 Evaluation . . . 7

3 Assignment 3 - WCET calculator **8**

Chapter 1

Assignment 1 — WCET

1.1 Assignment

After learning about scheduling and worst-case execution time (WCET) analysis, the first assignment focuses on developing a simple real-time application and analyzing its scheduling behavior and WCET. A passenger protection system is a system that has strong requirements for real time and fault tolerance. The following tasks should be completed:

- Implement a simplified version of the Airbag Deployment Algorithm described in [1]. Using either C or C++, Don't use floating point numbers or OO features.
- Analyze the WCET using IAR embedded workbench.
- Use BoundT to analyze the WCET. compare both results.

1.2 Implementation

The provided paper and lecture slides provide a starting point and outline how to implement the airbag deployment algorithm. It works by consistently calculating the current change in velocity as well as the length of the acceleration graph. Based in these values the system enters a STANDBY state and fires the airbag if the right conditions are met. Three thresholds are defined:

- ΔV_1: The change in velocity must be greater than ΔV_1 to enter the STANDBY state.
- ΔV_2: If change in velocity exceeds this value the airbag is fired immediately. ΔV_2 must be greater than ΔV_1.
- L: The length of the acceleration graph must be greater than L during standby state to fire the airbag.

Before implementation the algorithm was broken down into steps and states as follows:

1.2.1 Initialization

Sliding windows are initialized as arrays of size N and filled with zeros. The system starts in the *NORMAL* state. Everything after this is executed in an endless loop. Rudimentary timing is implemented using `usleep(1000000 / SAMPLES_PER_SECOND)`.

```
    // Initialize arrays for storing time samples
    int t1[T_SIZE1] = {0}; // Array for first time series
    int t2[T_SIZE2] = {0}; // Array for second time series

    // Define thresholds for change in velocity
    int deltaV1 = 20; // Threshold for entering standby state
    int deltaV2 = 50; // Threshold for firing the airbag
    int LThreshold = 300; // Threshold for length of acceleration graph

    // Variables for current calculations
    int deltaV = 0; // Current change in velocity
    int L = 0; // Current length of acceleration graph

    // Variable to hold the current sample index
    int currentSample;

    // State of the airbag system
    AirbagState state = NORMAL; // Initial state is NORMAL
```

1.2.2 Data Acquisition

Query new data, random values in this case, and update the sliding windows by shifting existing values and adding the new data. This step happens regardless of state.

```
    static void processSample(int t1[], int size1, int t2[], int size2, int *deltaV,
     int *L)
    {
    int currentSample = rand() % (10 + 1 + 30) - 30;
    printf("Current Sample: %d\n", currentSample);

    slideWindow(t1, size1, currentSample);
    slideWindow(t2, size2, currentSample);

    *deltaV = determineDeltaV(t1, size1);
    *L = determineL(t2, size2);

    printf("DeltaV: %d, L: %d\n", *deltaV, *L);
}
```

1.2.3 Calculation

Calculate ΔV and L within the scope of the sliding windows, using the formulas presented in the paper.

```
    static int determineDeltaV(int window[], int size)
    {
    double deltaV = 0;

    for (int i = 0; i < size; i++)
```

```
    {
        deltaV += window[i] * 1.0 / SAMPLES_PER_SECOND;
    }
    return round(deltaV);
}

static int determineL(int window[], int size)
    {

    double l = 0, dy = 0;
    double dx = 1.0 / SAMPLES_PER_SECOND;
    for (int i = 0; i < size - 1; i++)
    {
        dy = window[i + 1] - window[i];
        l += sqrt(pow(dy, 2) + pow(dx, 2));
    }
    return (int)l;
}
```

1.2.4 State Transition

Possible states are *NORMAL*, *STANDBY* and *FIRE*. If ΔV exceeds ΔV_1 the system enters the standby state. If ΔV exceeds ΔV_2 the system fires immediately. If the system is in STANDBY state and L exceeds the threshold L, the system fires. If the system is in STANDBY state and ΔV drops below ΔV_1 the system goes back to normal. For clarity these are implemented as switch cases.

Possible transitions are:

- NORMAL → STANDBY → NORMAL
- NORMAL → STANDBY → FIRE

```
        case NORMAL:
            processSample(t1, T_SIZE1, t2, T_SIZE2, &deltaV, &L);
            if (abs(deltaV) > deltaV1)
            {
                state = STANDBY;
            }
            break;

        case STANDBY:
            processSample(t1, T_SIZE1, t2, T_SIZE2, &deltaV, &L);
            if (abs(deltaV) > deltaV2 || L > LThreshold)
            {
                state = FIRE;
            }
            if (abs(deltaV) < deltaV1)
            {
                state = NORMAL;
            }
            break;

        case FIRE:
            exit(0);
            break;
```

1.2.5 Challenges

While the algorithm was simple to implement, upon trying to run it in the provided version of IAR Embedded Workbench didn't compile due to unsupported libraries and C features used. Many rewrites were necessary to fix it.

1.3 Evaluation

Describe tests and results.

Chapter 2

Assignment 2 — Scheduling policies

2.1 Assignment

2.2 Approach

2.3 Evaluation

Chapter 3

Assignment 3 - WCET calculator

[1] S.M. Mahmud and A.I. Alrabady. "A New Decision Making Algorithm for Airbag Control". *IEEE Transactions on Vehicular Technology* 44.3 (Aug. 1995), pp. 690–697. DOI: 10.1109/25.406638.